

DATA 479 Final Project Report

Group 5

Matthew Tran (30212948)

Eva Caton (30112095)

Yurii Bezborodov (30114669)

Asjad Zubair (30147898)

For this project, we chose to focus on analyzing climate changes over the span of 3 years across 3 decades (the subset of the data we chose covers the years 1999-2001, 2009-2011, and the years in 2019-2021) for 40 different countries across the globe with varying climate conditions, including regions with hotter climates (e.g. Egypt, Dubai) and colder climates (e.g. Antarctica, Canada), tropical regions (e.g. Hawaii), regions more prone to natural disaster (e.g. Japan, Philippines), as well as regions with milder/warm climates (e.g. Midwest US, Scandinavian countries). The list of all countries (and the station IDs used) can be found in the *stations.txt* file located in the *task1* folder.

Task 1 - Cloud Data Ingestion & Exploratory Data Analysis:

Matthew Tran

There are a total of **127,839** records across all 9 years for all 40 stations in the dataset. It is important to note that although the number of total records seems lower than expected (*one might expect to see about* $9 \times 40 \times 365 = 131,400$ *records total*), not every single date is recorded in the dataset, and some stations like in Pakistan (*Station 41515099999*) for some years, only record a few times a month, for example in 2000, only 12 dates in total were recorded at the Pakistan station.

The schema and data types from this dataset read from PySpark are as follows, which describes the column name, data type of column, and whether the column can store NULL values (*i.e. if the column is nullable*):

Column Name	Data Type	Nullable
STATION	string	true
DATE	date	true
LATITUDE	double	true
LONGITUDE	double	true
ELEVATION	double	true
NAME	string	true
TEMP	double	true
TEMP_ATTRIBUTES	double	true
DEWP	double	true
DEWP_ATTRIBUTES	double	true
SLP	double	true
SLP_ATTRIBUTES	double	true
STP	double	true
STP_ATTRIBUTES	double	true
VISIB	double	true
VISIB_ATTRIBUTES	double	true
WDSP	double	true
WDSP_ATTRIBUTES	double	true
MXSPD	double	true
GUST	double	true
MAX	double	true
MAX_ATTRIBUTES	string	true
MIN_ATTRIBUTES	double	true
MIN	double	true
MIN_ATTRIBUTES	string	true
PRCP	double	true

PRCP_ATTRIBUTES	string	true
SNDP	double	true
FRSHTT	integer	true

The dataset in terms of data quality, on inspection, initially contained a few completely null records when loaded from Azure which did not contribute to the dataset, hence was filtered out to prevent skewed counts in the number of records, as well as the number of weather stations in the dataset. Upon further analysis, we find that the data contained only a few records with missing/invalid values (*labeled with the value '9999.9'*) in either the MIN (*minimum temperature recorded*) or MAX (*maximum temperature recorded*) columns for temperature fields (*no missing values were found in the TEMP (average temperature recorded) column*) for only 10 stations in the dataset (a total of 20 records were found to contain missing values).

Task 2 - Distributed Analytics with PySpark

Yurii Bezborodov

The subset of the dataset used.

For Task 2, we used a subset of the NOAA GSOD dataset that had already been uploaded to Azure Blob Storage in Task 1. Our subset included daily weather data from the years 1999, 2000, 2001, 2009, 2010, 2011, 2019, 2020, and 2021. These files contained data for many different weather stations, so the subset met the project requirement of using multiple years and a sufficient number of stations. For this task, we focused only on the fields needed for the analysis: STATION, DATE, and TEMP.

Transformations used.

The data was loaded into PySpark from Azure Blob Storage, where the files were organized by year. After loading the data, we selected only the columns needed for this task: STATION, DATE, and TEMP. We then cleaned the data by removing missing or invalid temperature values. After that, we extracted the year from the DATE column and grouped the data by station and year. Using PySpark aggregation, we calculated the average annual temperature for each weather station. The final output was formatted as: station, year, average_temperature.

Why Spark is suitable for this task.

Spark is a good choice for this task because the GSOD dataset is large and split into many files. Instead of opening and processing each file manually, PySpark can read them together and handle the computation more efficiently. It also makes it easier to clean the data and perform grouped calculations on a large dataset using distributed processing. Overall, Spark is well suited for this kind of climate analysis because it is faster, more scalable, and much more convenient for working with large collections of data files.

Task 3 - Distributed Analytics with Hadoop MapReduce

Asjad Zubair

Advantages and Limitations

Hadoop MapReduce excels in fault tolerance, as it is designed to handle node failures. If a task fails, Hadoop automatically retries it on another node, making it better suited for jobs with long runtimes. MapReduce also scales horizontally to more nodes, and has strict separation of the map, shuffle/sort, and reduce phases which ensures manageable data flow at scale.

A limitation of MapReduce is the disk overhead. MapReduce writes intermediate results to storage between the map and reduce phases. For multi-step jobs, this results in repeated disk reads and writes, which makes the process significantly slower than alternatives that work with memory.

Speaking of in-memory processing, this is a significant advantage of Apache PySpark. Processing data in memory instead of reading and writing constantly to disk avoids the disk overhead of MapReduce, and makes computations and multi-step jobs significantly faster.

In-memory processing has its own limitations. PySpark requires significantly more memory than MapReduce due to this, and memory constraints can quickly lead to performance losses.

Code Complexity

The same task was implemented in both frameworks, calculating the average annual temperature per station. The PySpark implementation from Task 2 took

approximately 15 lines for the computation, whereas the MapReduce implementation from Task 3 required two separate scripts and totalled approximately 80 lines of code.

The expectation from this would be that the PySpark implementation would execute faster than the MapReduce implementation. However, contrary to that expectation, the opposite is true. The PySpark implementation executed in 113.59 seconds whereas MapReduce took only 74.21 seconds to run. There are some possible explanations for why this may have been the case.

First of all, the PySpark implementation reads data directly from Azure Blob storage over the network, which introduces latency to every read and write. The MapReduce method processed the data after it had already been stored in the local HDFS, avoiding this latency. It is also possible that the extra time taken by PySpark was partly due to the processing overhead that PySpark has. The primary performance advantage of PySpark grows with job complexity. Since this is a relatively simple task, it is possible that the advantage was not prevalent enough for PySpark to outperform MapReduce.

Task 4 - Climate Analytics Report

Eva Caton

Analyses:

Station Comparison

Temperature Variability

What I noticed in the station comparison is that as the stations move further south the temperatures steadily increase. However, this can not be said for Russia, which actually held the lowest average temperature value. As well, I found an interesting point that Nevada in America held temperatures equivalent to countries that were much farther south. I'm curious if this anomaly would also be seen if we looked at a larger variety of cities within other countries. The relatively small changes to the standard deviation were quite surprising to find considering the large distribution of the years chosen. Though, it is still a finding that the standard deviation remained statistically insignificant, as this means that the variability of the weather has not grown to outrageous proportions.

These findings matter because they aid us in our analysis of weather. Knowing where to expect certain climates and the ability to have greater confidence when predicting future standard deviation is vital for valid statistical inference. This allows for probable prediction of future weather events and allows us to spot outliers as we have a strong foundation of references to compare to.

Team Contributions:

Matthew Tran - Task 1

Yurii Bezborodov - Task 2

Asjad Zubair - Task 3

Eva Caton - Task 4